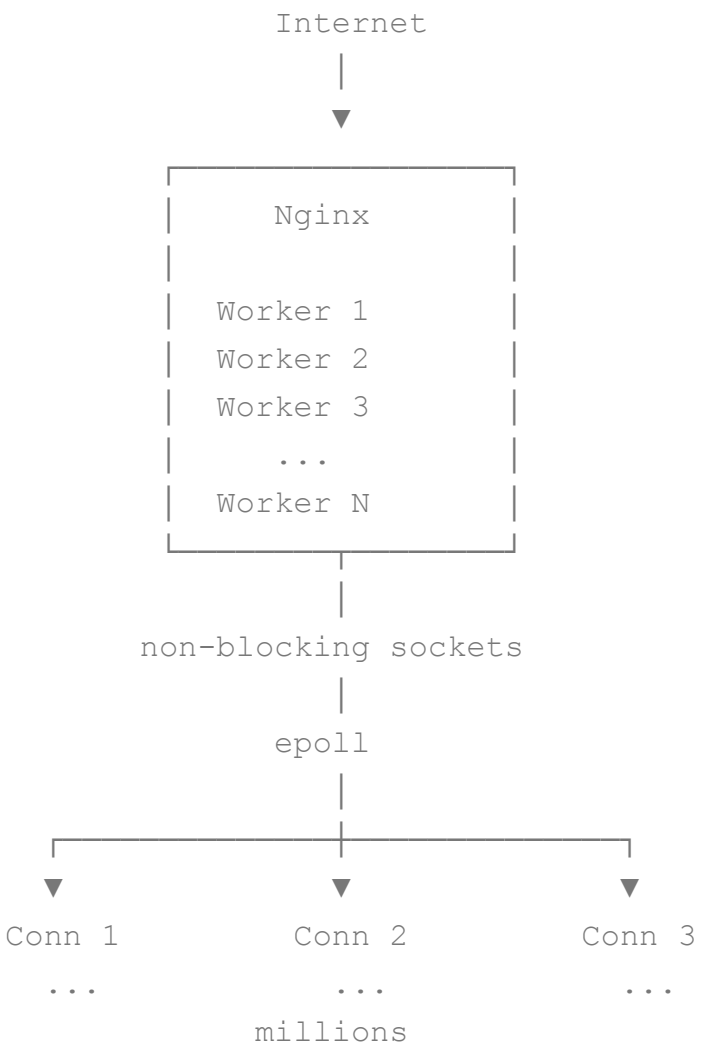
Nginx scales to millions of concurrent connections because it combines **Linux `epoll` + non-blocking I/O + event-driven workers + efficient connection state + horizontal scaling**. `epoll` is only one piece of the architecture.

The key mental model is:



Let's trace how this works.

---

1. Why millions of connections are possible

Suppose Nginx has:

```
1,000,000 TCP connections
```

but only:

```
5,000 connections
```

are actively doing I/O at a particular instant.

Nginx doesn't need:

```
1,000,000 threads
```

and it doesn't need to continuously scan:

```
FD 1
FD 2
FD 3
...
FD 1,000,000
```

Instead:

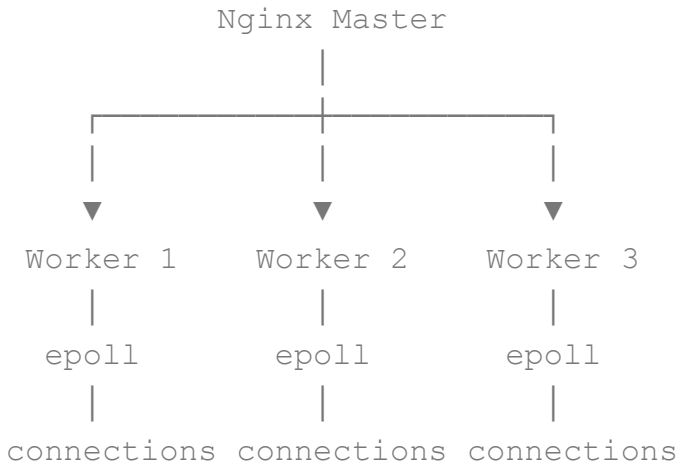
```
1,000,000 connections
        |
        ▼
      epoll
        |
        ▼
5,000 ready events
        |
        ▼
Nginx event loop
```

This is the fundamental scalability property.

---

2. Nginx uses an event-driven worker architecture

A simplified Nginx process architecture looks like:



The **master process** primarily manages configuration, lifecycle, and worker processes.

The workers do the actual request processing.

A typical configuration conceptually looks like:

```
worker_processes auto;
```

This allows Nginx to use an appropriate number of workers, commonly related to available CPU cores.

---

3. One worker can manage huge numbers of connections

Consider:

```
CPU cores = 16
```

Nginx might have approximately:

```
16 workers
```

Each worker can manage many connections.

For example:

```
Worker 1
 ├── 60K connections
```

```
  ├── epoll
  └── event loop

Worker 2
  ├── 65K connections
  ├── epoll
  └── event loop

...

Worker 16
  ├── 70K connections
  ├── epoll
  └── event loop
```

The exact connection capacity is not determined simply by CPU count; memory, kernel limits, network bandwidth, TLS, request rates, upstreams, and workload all matter.

---

4. What happens when a client connects?

Let's follow one connection.

```
Client
  │
  │ TCP SYN
  ▼
Nginx listening socket
  │
  ▼
accept()
  │
  ▼
new client socket
  │
  ▼
non-blocking mode
  │
  ▼
register with event mechanism
  │
```

```
   ▼
epoll
```

The worker now has a connection object associated with the socket.

Conceptually:

```
FD 7231
   │
   ▼
Nginx Connection
   ├── socket
   ├── read state
   ├── write state
   ├── buffers
   ├── HTTP state
   └── timers
```

This is vastly cheaper than creating a dedicated thread for every connection.

---

5. The connection is mostly idle

This is the key reason event-driven servers scale.

Imagine:

```
1,000,000 keep-alive connections
```

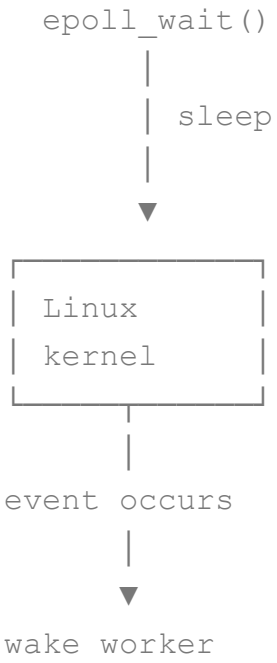
Most clients aren't sending data right now.

The state might look like:

```
Conn 1      IDLE
Conn 2      IDLE
Conn 3      IDLE
...
Conn 999999 IDLE
Conn 1000000 IDLE
```

Nginx doesn't actively execute code for every idle connection.

Instead:



The worker consumes CPU primarily when something needs processing.

---

6. Client sends a request

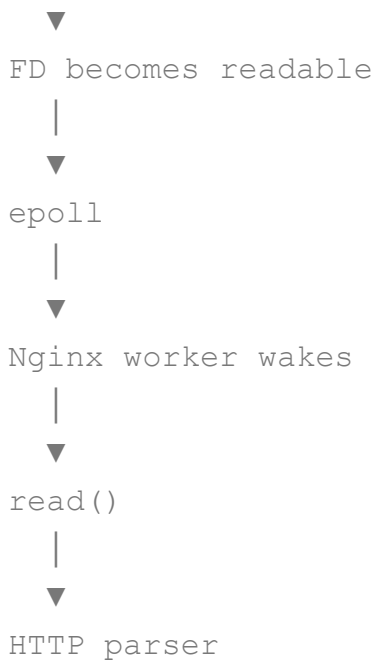
Suppose:

```
GET /index.html HTTP/1.1
```

arrives.

The flow becomes:

```
Client
  |
  ▼
NIC
  |
  ▼
Linux TCP/IP stack
  |
  ▼
Socket receive buffer
  |
```



So:

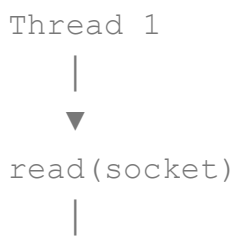


---

7. Nginx doesn't block waiting for a socket

This is crucial.

A traditional blocking server might do:



```
    | BLOCK
    |
    | waiting for network
    |
    ▼
data arrives
```

Nginx instead uses non-blocking I/O:

```
Worker
  |
  ▼
epoll_wait()
  |
  | sleep until useful work
  ▼
FD ready
  |
  ▼
read()
```

The worker isn't wasting a thread per idle connection.

---

8. What happens if the request isn't complete?

Suppose TCP delivers:

```
GET /users/
```

but the rest arrives later:

```
123 HTTP/1.1
```

Nginx doesn't block.

It maintains connection state:

```
Connection
 ├── request buffer
 │       │
 │       └── "GET /users/"
 │
 └── waiting for more data
```

Then it returns to:

```
epoll_wait()
```

Later:

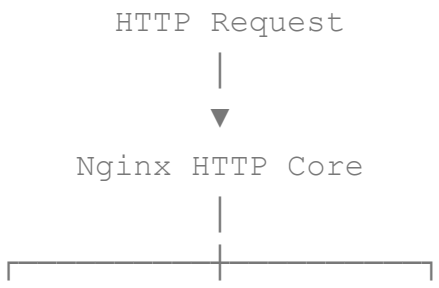
```
more TCP data
      │
      ▼
   EPOLLIN
      │
      ▼
   read()
      │
      ▼
append to buffer
      │
      ▼
HTTP request complete
```

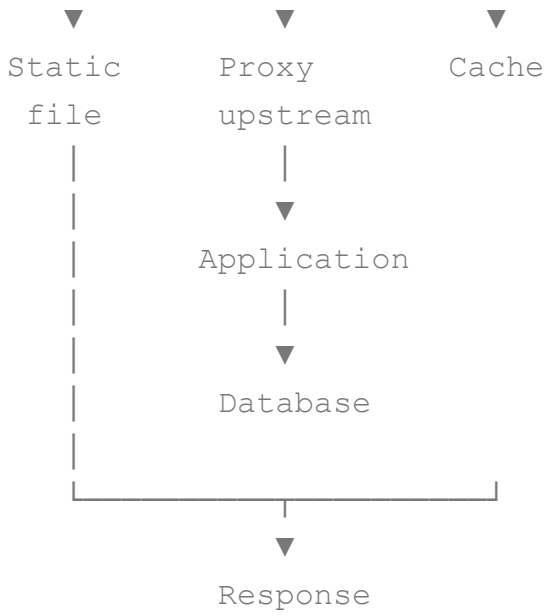
This is one of the major differences between event-driven and thread-per-connection architectures.

---

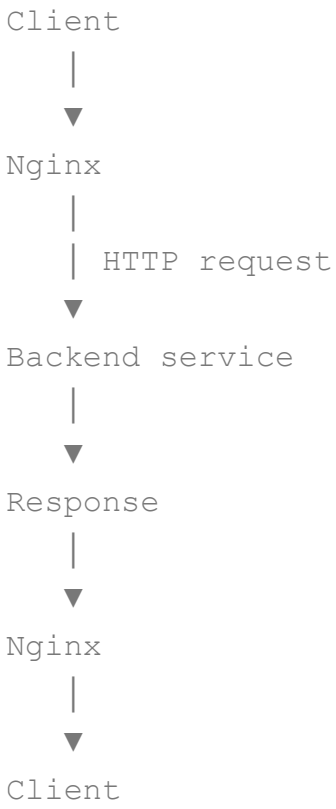
9. Request processing

Once the request is complete:





For a reverse proxy:



---

10. Nginx also uses event-driven I/O for upstreams

This is a major reason Nginx is powerful as a reverse proxy.

Suppose Nginx has:

```
100,000 clients
```

and requests go to:

```
Backend service
```

Nginx may have thousands of simultaneous upstream connections.

It can monitor both:

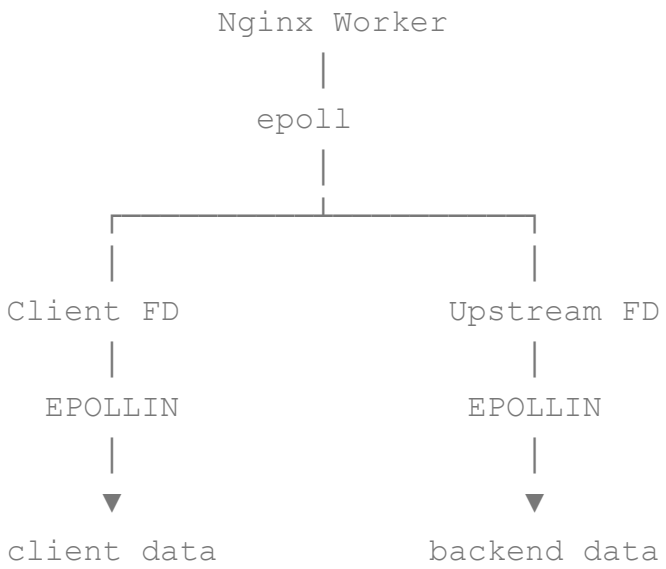
```
Client socket
```

and:

```
Upstream socket
```

using the event mechanism.

Conceptually:

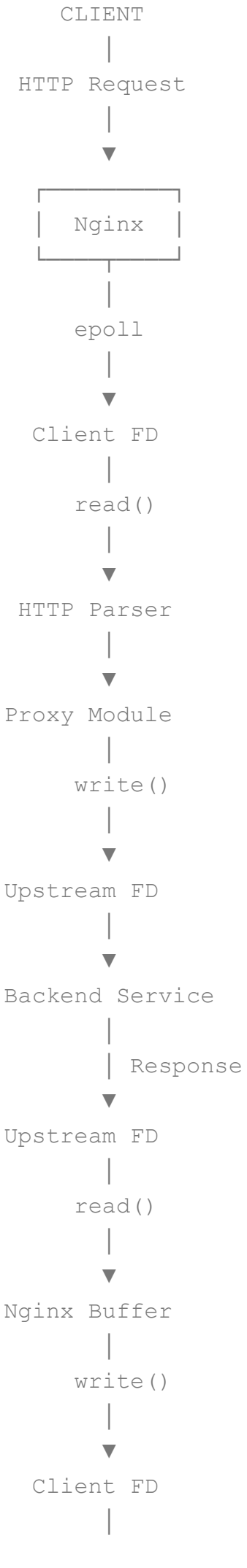


No dedicated thread is required for each client/backend pair.

---

11. Full reverse-proxy flow

This is worth memorizing:


CLIENT
HTTP Request
Nginx
epoll
Client FD
read()
HTTP Parser
Proxy Module
write()
Upstream FD
Backend Service
Response
Upstream FD
read()
Nginx Buffer
write()
Client FD



Both sides are asynchronous from the worker's perspective.

---

12. Why `EPOLLOUT` matters

Suppose Nginx receives a 20 MB response from an upstream.

The client might be slow.

You can't assume:

```
write(client_fd, 20MB)
```

will succeed completely.

It might only accept:

```
64 KB
```

So Nginx maintains buffered data.

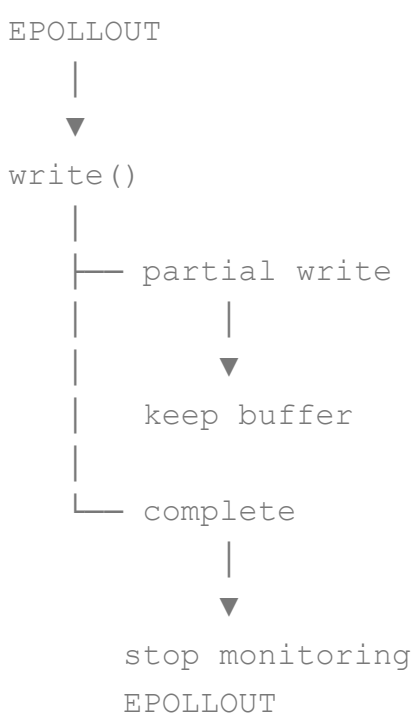
```
Backend
   |
   | 20 MB
   ▼
Nginx response buffer
   |
   | 64 KB
   ▼
Client socket
```

Remaining:

```
19.936 MB
```

Nginx waits until the socket is writable again.

Conceptually:



This is critical for handling **slow clients without blocking workers**.

---

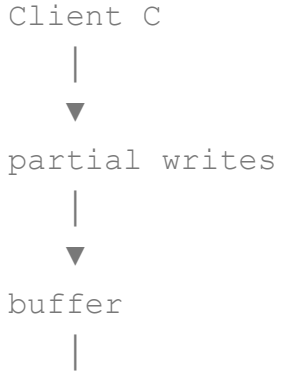
13. Slow clients are particularly important

Imagine:

```
Client A → 1 Gbps
Client B → 1 Mbps
Client C → 10 Kbps
```

Nginx shouldn't allow Client C to block a worker.

Instead:



```
    ▼
EPOLLOUT
    |
    ▼
continue later
```

Meanwhile:

```
Client A ──► processed
Client B ──► processed
Client C ──► waiting
```

The event loop can serve other connections.

---

14. Why CPU isn't proportional to connection count

This is one of the most important concepts.

Suppose:

```
Connections = 1,000,000
Requests/sec = 10,000
```

CPU work is primarily related to:

```
active events
+
request processing
+
network processing
```

rather than simply:

```
1,000,000 connections
```

So:

```
1M mostly-idle connections
```

can be much easier than:

```
100K connections
with millions of requests/sec
```

Connection count alone isn't a sufficient capacity metric.

---

15. Memory becomes a major constraint

At million-connection scale, memory can become more important than CPU.

Each connection requires some state:

```
Connection
 ├── socket/kernel state
 ├── Nginx connection structure
 ├── HTTP request state
 ├── buffers
 ├── timers
 └── optional TLS state
```

Therefore:

```
1 connection → X KB
1,000,000 connections → approximately X GB
```

The actual memory footprint depends heavily on configuration and workload.

This is why high-scale connection testing must measure:

```
RSS
kernel memory
socket memory
buffer usage
TLS memory
connection structures
```

---

16. File descriptor limits

Linux processes have file-descriptor limits.

You can't simply say:

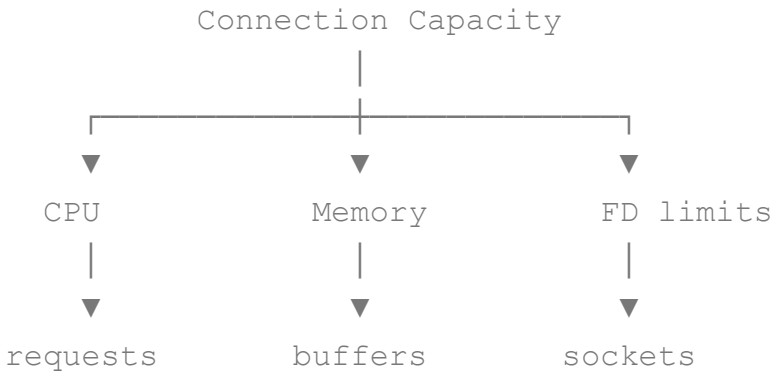
```
"I want 10 million connections."
```

and expect it to work.

You need to consider:

```
ulimit
RLIMIT_NOFILE
kernel file table limits
socket memory
ephemeral ports
listen backlog
network capacity
```

Conceptually:



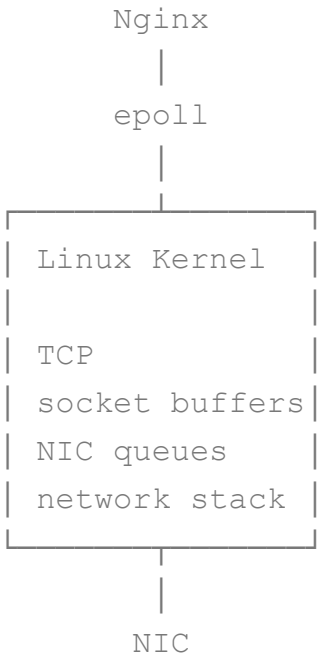
---

17. Kernel networking also matters

The architecture isn't:

```
Nginx → epoll → millions
```

There is a large kernel subsystem underneath:



At very high scale, you have to tune and understand both userspace and kernel networking.

---

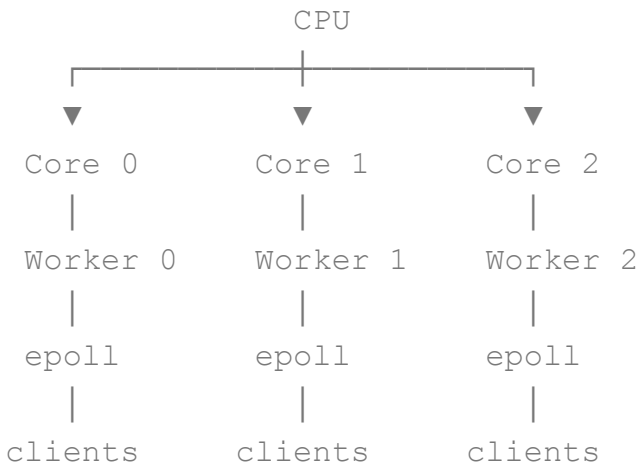
18. Multi-core scaling

Suppose:

```
32 CPU cores
```

Nginx can run multiple workers.

Conceptually:



This gives Nginx parallelism without requiring a thread for every connection.

---

19. Worker isolation

A useful property of Nginx's architecture is that workers can operate largely independently.

Conceptually:

```
Worker 1
 ├── event loop
 ├── connections
 └── state

Worker 2
 ├── event loop
 ├── connections
 └── state

Worker 3
 ├── event loop
 ├── connections
 └── state
```

This reduces the need for locks around every connection operation.

That is an important LLD/HLD principle:

> **Prefer ownership of connection state by an event loop over shared mutable state across workers.**

---

20. But there is a subtle accept problem

Multiple workers may be interested in incoming connections.

Nginx has mechanisms to coordinate worker access to the listening socket and Linux provides facilities such as `SO_REUSEPORT` that can be used in appropriate architectures.

The goal is:

```
incoming connections
        |
```

```
        ▼
distributed across workers
```

rather than:

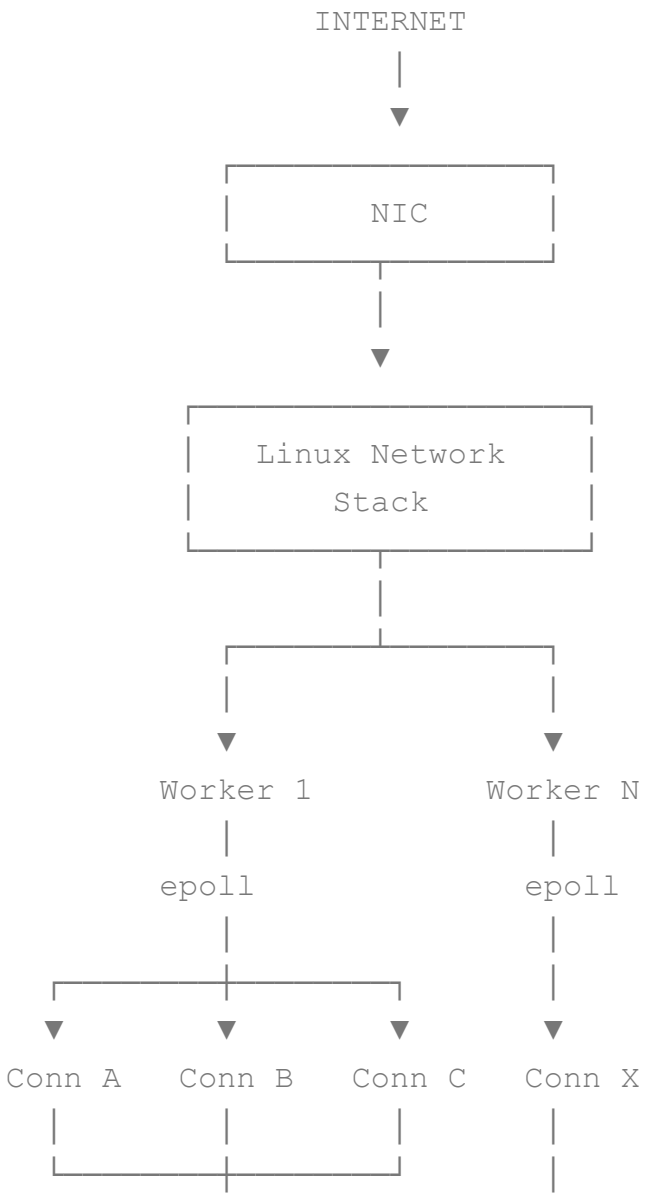
```
Worker 1
   ↑
all connections
```

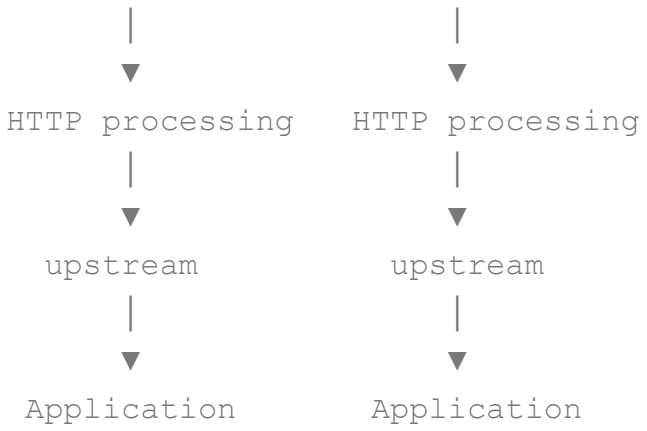
which would create a bottleneck.

---

21. The complete Nginx architecture

Putting everything together:





---

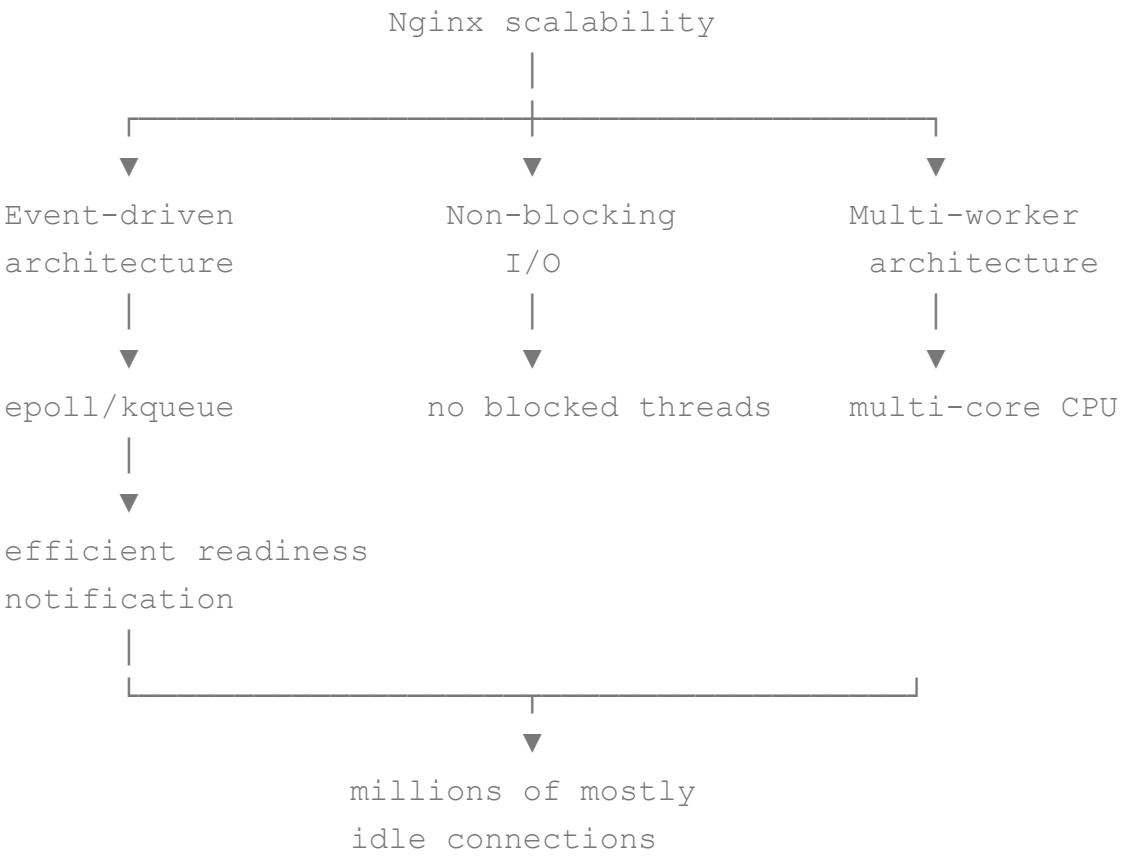
22. Where the scalability actually comes from

It's tempting to say:

> "Nginx scales because it uses `epoll`."

That's incomplete.

The actual chain is:



Then additional scalability comes from:

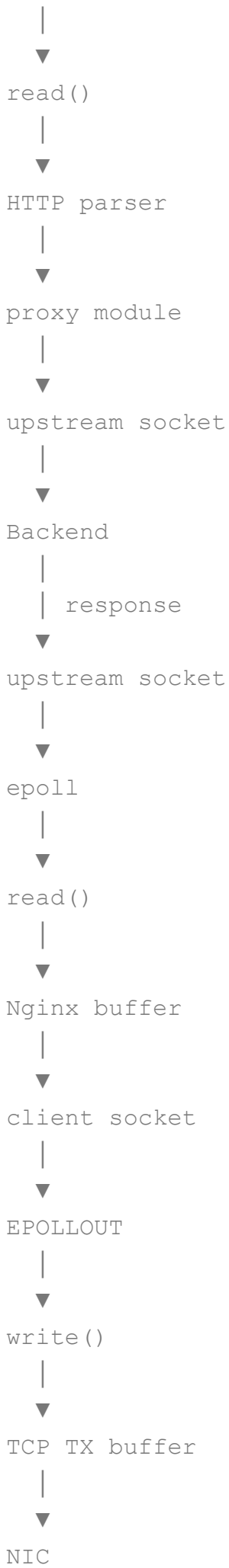
```
horizontal scaling
connection reuse
buffer management
caching
upstream keep-alive
TLS optimization
kernel/network tuning
rate limiting
timeouts
backpressure
```

---

23. One request through Nginx

Let's condense everything into one sequence:

```
CLIENT
  |
  |  GET /api/users/123
  ▼
TCP
  |
  ▼
NIC
  |
  ▼
Linux TCP stack
  |
  ▼
socket RX buffer
  |
  ▼
FD ready
  |
  ▼
epoll
  |
  ▼
epoll_wait() returns
  |
  ▼
```


Nginx worker
read()
HTTP parser
proxy module
upstream socket
Backend
response
upstream socket
epoll
read()
Nginx buffer
client socket
EPOLLOUT
write()
TCP TX buffer
NIC



24. The most important interview insight

If an interviewer asks:

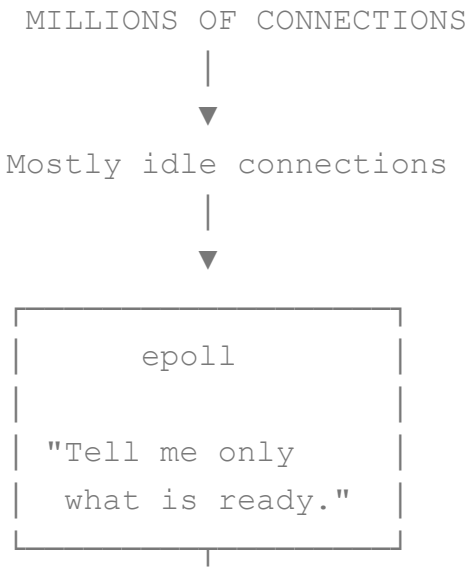
> **"How can Nginx handle millions of connections with only a relatively small number of workers?"**

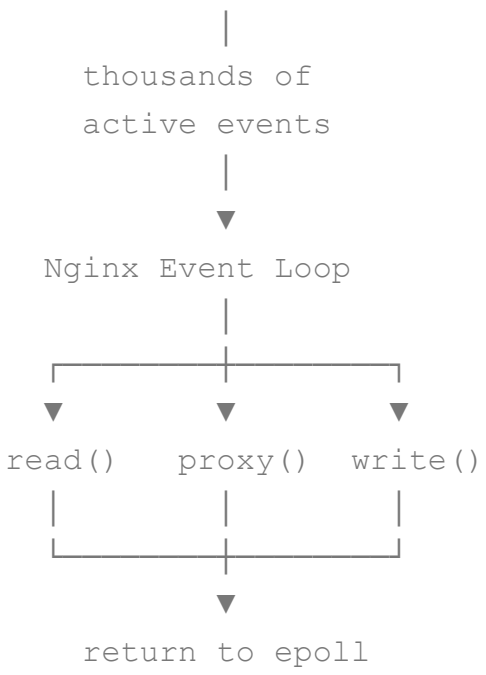
A strong answer is:

> Nginx uses an event-driven architecture where worker processes use non-blocking sockets and an OS event-notification mechanism such as Linux `epoll`. Instead of dedicating a thread to every connection or repeatedly scanning all connections, a worker sleeps in the event loop until the kernel reports sockets that are ready for I/O. Each worker maintains lightweight connection state and processes only active events. Multiple workers utilize multiple CPU cores, while horizontal scaling distributes connections across machines. The practical connection limit is then determined by the combined constraints of CPU, memory, file descriptors, kernel socket resources, network bandwidth, TLS overhead, request rate, and workload—not by `epoll` alone.

That answer demonstrates **DSA + Linux internals + LLD + HLD** rather than simply saying "`epoll` is fast."

One final mental model





The core scalability trick is not "millions of connections are processed simultaneously." It is that millions of connections can exist as mostly dormant state while a small number of workers efficiently process the subset that currently has work to do.